Contents

List of Figures

List of Tables

Chapter 1

Introduction

Chapter 2

Matching Problems and Algorithms

2.1 Definitions

Let $G = (V, E)$ be an undirected graph with $|V| = n$ vertices and $|E| = m$ edges. We call G bipartite if we can partition its vertex set into two sets V_1, V_2 such that for each edge $u, v \in E$, $u \in V_1$ and $v \in V_2$. If every vertex of V_1 is adjacent to every vertex of V_2 and $|V_1| = n_1, |V_2| = n_2$, then we call G a complete bipartite graph and denote it by K_{n_1,n_2}. By a path, we mean a simple path, i.e., a path in which no vertex is repeated. The length of a path is the number of edges it contains.

From set theory, we recall that the symmetric difference of two sets A and B, denoted by $A \triangle B$, is the set of elements that belong to A or B but not both. Thus, $A \triangle B = (A \backslash B) \cup (B \backslash A) = (A \cup B) \backslash (A \cap B)$. Also, if we want to add an element x to set A, we shall use the shorthand $A + x \coloneqq A \cup \{x\}$, and if we to remove it $A - x \coloneqq A \backslash \{x\}$.

We write $T(n) = O(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \leqslant cf(n)$ for all $n \geqslant n_0$. Most of the time, $T(n)$ will denote a bound on the worst-case running time of an algorithm defined on the positive integers. Similarly, we say that $T(n) = \Omega(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \geqslant cf(n)$ for all $n \geqslant n_0$. Finally, we say that $T(n) = \Theta(f(n))$ if and only if $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$.

A matching of a graph $G = (V, E)$ is a subset of E such that each vertex is incident to at most one edge of the subset. The cardinality of a matching is the number of edges it contains. A vertex is called matched if it is incident to an edge of M. Otherwise, it is called unmatched or free. A matching is called maximal if no edge can be added without violating the matching property. A matching is maximum if no other matching has more edges. Clearly, every maximum matching is maximal (the converse is not always true). The following two definitions are fundamental in the study of matchings.

Definition 2.1.1. *A path P in G with respect to a matching M is called alternating if its edges are alternatingly out of and in M.*

Definition 2.1.2. *A path P in G with respect to a matching M is called augmenting if it is alternating and its endpoints are free.*

It is straightforward to see that every augmenting path has odd length. We will use the term M-augmenting whenever we want to emphasize the matching M.

2.2 Maximum Bipartite Matching

In this section, we present two famous algorithms for finding a maximum bipartite matching and prove their correctness.

The first algorithm is derived from a theorem of Berge [Ber57], previously observed by Petersen [Pet91].

Theorem 2.2.1. *Let $G = (V, E)$ be a graph and let M be a matching in G. Then M is maximum if and only if there is no M-augmenting path in G.*

Proof. Suppose that M is maximum. For the sake of contradiction, assume that there exists an M-augmenting path P with u and v as its endpoints in G. Then we can see that the matching $M^{'} = M \triangle E(P)$ is a valid matching in G and satisfies $|M^{'}| = |M| + 1$. Indeed, if we remove all edges of P that belong to M and add the remaining ones to M, we add one more edge to M in total (because P has odd length). The new matching is valid because u and v were free, and they were not incident to any matching edge, and every internal vertex of P remains incident to exactly one edge. All vertices outside P are unaffected. As a result, the new matching $M^{'}$ is valid and its cardinality is greater than that of M. This is a contradiction.

Now suppose that G does not contain any M-augmenting path and that a matching $M^{'}$ exists in G such that $|M^{'}| > |M|$. Consider the graph $M_0 = M \triangle M^{'}$. Each vertex in M_0 may be matched in both matchings or in either one. So, every vertex has degree at most 2. Hence, every component of M_0 is either a path or a cycle of even length (otherwise, there would exist a vertex that is incident to two edges in the same matching). Because M_0 has more edges of $M^{'}$ than of M, this also means that there exists a path P with more edges of $M^{'}$ than of M, and because the edges are alternating, P is an M-augmenting path, which contradicts our assumption that G does not contain any M-augmenting path. □

Augmenting-Path Algorithm. Based on the theorem above, we can write an algorithm that will try to find an augmenting path, alternate its edges (this will increase the size of the current matching by one), and repeat the whole process until we can't find any augmenting path. $N(v)$ represents all the vertices u such that $\{v, u\} \in E(G)$.

Algorithm 2.1 Augmenting Path Algorithm

Require: Graph $G = (V, E)$, vertex v
Ensure: Array matched such that $matched[u] = v$ and $u \in V_2$ and $v \in V_1$ if and only if $\{u, v\} \in E(G)$ belongs to the matching

```
1: function DfsAugmentingPath(int v, bool visited[], int matched[])
2:     if visited[v] then
3:         return FALSE;
4:     end if
5:     visited[v] ←TRUE;
6:     for all u ∈ N(v) do
7:         if matched[u] = ∅ or DfsAugmentingPath(matched[u], visited, matched) then
8:             matched[u] ← v;
9:             return TRUE;
10:        end if
11:    end for
12:    return FALSE;
13: end function
```

Algorithm 2.1 is getting called from all vertices in the left part of bipartite graph (let's call it V_1). We will use an extra array called visited to track the vertices from V_1 that we have already visited across the path and we have to reset it to be false for all these vertices before the initial call. Array matched marks the matched vertices only from the right part of the bipartite graph (let's call it V_2). To get the edges of the matching, we simply take all the edges $\{v, matched[v]\}$ such that $matched[v]$ is not empty and store them to set *matching*.

Algorithm 2.2 Maximum Bipartite Matching in $O(nm)$

Require: Bipartite graph $G = (V_1 \cup V_2, E)$
Ensure: Array *matching* that corresponds to the edges of the maximum matching

```
1: function MainAugmentingPath(int v, bool visited[], set matching)
2:     for all v ∈ V2 do
3:         matched[v] ← ∅;
4:     end for
5:     for all v ∈ V1 do
6:         for all u ∈ V1 do
7:             dist[i] ← FALSE;
8:         end for
9:         DfsAugmentingPath(v, visited, matched);
10:    end for
11:    matching ← ∅;
12:    for all v ∈ V2 do
13:        if matched[v] ≠ ∅ then
14:            matching ← matching ∪ {{matched[v], v}};
15:        end if
16:    end for
17:    return matching;
18: end function
```

The idea is that if the initial call is from a free vertex, say v, and we cannot find another

free vertex u such that the edge $\{v, u\}$ exists, then we simply try to find a path (v, x, y) such that $\{v, x\}$ is not in the matching, but $\{x, y\}$ is, so we can start trying again from the vertex y, until we find a free vertex and then recursively alternate the path.

Lemma 2.2.2. *If we call Algorithm 2.1 from a free vertex v and augmenting path with endpoint v exists, then it will return* TRUE *and alternate its edges.*

Proof. Suppose that a free vertex $u \in V_2$ exists such that $\{v, u\} \in E(G)$. Then from line 7 the algorithm successfully finds an augmenting path. Now suppose that such a vertex doesn't exist and $P = (v, v_1, v_2, ..., u)$ any augmenting path with endpoint the vertex v. Since P is alternating, the edge $\{v, v_1\}$ does not belong to the matching, while $\{v_1, v_2\}$ does. Because the algorithm checks all neighbors of v (line 6), it will eventually examine v_1 (assuming that all the other neighbors - if they exist - return false), and line 7 does a recursive call for $matched[v_1] = v_2$ (note that v_2 is impossible to be visited, otherwise an alternative path $(v, ..., v_2)$ exists and the algorithm would have already found another augmenting path from that recursive call to u or to another free vertex). Repeating the same argument along P, the algorithm eventually reaches the free vertex $u \in V_2$, where line 7 is true since $matched[u] = \varnothing$. The recursive calls then return TRUE and update the array matched accordingly, thus alternating the edges of P. □

From this, we get the following remark.

Remark 2.2.3. *After an initial call of Algorithm 2.1 from a free vertex $v \in V_1$, the only vertices that can become matched are v and one free vertex of V_2. If no augmenting path is found, then the matching remains unchanged.*

Then, we can prove the following lemma.

Lemma 2.2.4. *Every initial call to Algorithm 2.1 from Algorithm 2.2 takes a free vertex as parameter.*

Proof. This is obviously true in the first iteration. Assume that we are at iteration i, and that the statement holds for all previous iterations. Let v be the vertex from which we are about to initially call Algorithm 2.1. If v is free, then we are done. If it is not, v must have become matched during some previous iteration. However, by Remark 2.2.3, in each iteration the only vertex of V_1 that can become matched is the vertex from which Algorithm 2.1 was initially called. Since Algorithm 2.1 has never been initially called from v before iteration i, the vertex v could not have become matched earlier. □

The output of the Algorithm 2.2 (set matching) is always a valid matching. Since the algorithm initializes with an empty matching and only updates it by alternating edges along valid augmenting paths starting from free vertices (Lemma 2.2.2), the maintained set of edges is always a valid matching. Now we have to prove that it is maximum.

Theorem 2.2.5. *Algorithm 2.2 returns a maximum matching.*

Proof. Let $v_1, v_2, \ldots, v_k$ be the vertices of V_1 in the order in which Algorithm 2.2 processes them. For each $i \in \{0, 1, \ldots, k\}$, let M_i be the matching after the first i iterations of the algorithm, with $M_0 = \varnothing$. Also, let G_i be the subgraph of G induced by the vertex set

$$\{v_1, v_2, \ldots, v_i\} \cup V_2.$$

We will prove by induction on i that M_i is a maximum matching of G_i.

For $i = 0$, the graph G_0 has no vertices in V_1, so the empty matching M_0 is trivially maximum.

Now assume that M_i is a maximum matching of G_i, and consider iteration $i+1$, where the algorithm processes the vertex v_{i+1}. By Lemma 2.2.4, the initial call to Algorithm 2.1 is made from a free vertex. We consider two cases.

Case 1: *DfsAugmentingPath*(v_{i+1}) (Algorithm 2.1 with parameter the vertex v_{i+1}) returns TRUE. Then, by Lemma 2.2.2, the algorithm finds an augmenting path in G_{i+1} with endpoint v_{i+1} and alternates its edges. Therefore,

$$|M_{i+1}| = |M_i| + 1.$$

On the other hand, any matching of G_{i+1} can use at most one edge incident to the new vertex v_{i+1}. Hence, by removing that edge if it exists, we obtain a matching of G_i, and therefore every matching of G_{i+1} has size at most $|M_i| + 1$. Since M_i is maximum in G_i, it follows that M_{i+1} is a maximum matching of G_{i+1}.

Case 2: *DfsAugmentingPath*(v_{i+1}) returns FALSE. Then, again by Lemma 2.2.2, there is no M_i-augmenting path in G_{i+1} with endpoint v_{i+1}. Since M_i is maximum in G_i, Theorem 2.2.1 implies that there is no M_i-augmenting path contained entirely in G_i. Thus, there is no M_i-augmenting path in G_{i+1} at all, because any augmenting path in G_{i+1} that is not contained in G_i must involve the only new vertex of V_1, namely v_{i+1}. Therefore, by Theorem 2.2.1, M_i is already a maximum matching of G_{i+1}. Since the matching does not change in this case, we have

$$M_{i+1} = M_i,$$

and so M_{i+1} is a maximum matching of G_{i+1}.

In both cases, M_{i+1} is a maximum matching of G_{i+1}. This completes the induction. □

Before each search, we initialize the visited array which takes $O(n)$ time. We call *DfsAugmentingPath* for each vertex $v \in V_1$ searching for an augmenting path. To find an augmenting path, we will visit each vertex and each edge at most once, so the total complexity is $O(nm)$ when $m = \Omega(n)$.

Hopcroft-Karp Algorithm Hopcroft and Karp [HK73] provided a faster algorithm, again based on finding augmenting paths, but in a smarter way. The algorithm runs in phases. In each phase, we find a maximal set of vertex-disjoint shortest M-augmenting paths and take the symmetric difference of the current matching M with the union of these paths. The process stops when no augmenting path can be found.

Our implementation is based on the version of Shimon Even and Alon Itai, who made significant modifications to Dinic's algorithm for the maximum flow problem [Din70, Din06]. As we will see at the end of this section, maximum flow and maximum bipartite matching are closely connected problems.

The key idea of the algorithm is to efficiently find these paths and alternate their edges. To do that, we construct a new graph called the layered network. We describe this graph with an array called level, defined on the vertices of V_1. Initially, every free vertex of V_1 gets level 1. Then, if a vertex $u \in V_1$ has level i, every vertex of V_1 that can be reached from u by first taking an unmatched edge to a vertex of V_2 and then a matched edge back to V_1 gets level $i + 1$, provided it has not already received a level. The vertices are visited in Breadth-First Search (BFS) order, starting from all free vertices of V_1. In this way, *level*[v]

represents the minimum alternating distance of $v \in V_1$ from the set of free vertices of V_1 in the current phase. If all edges in the layered network are directed from a source vertex in V_1 to a destination vertex in V_2, it is clear that the resulting graph is a dag (directed acyclic graph). It is important to emphasize that two

Algorithm 2.3 BFS Layered Network Construction

Require: Bipartite graph $G = (V_1 \cup V_2, E)$, arrays *next*, *prev* represent the current matching
Ensure: Array *level* for the vertices of V_1 describes the layered network

```
1: function BfsLayeredNetwork(int next[], int prev[], int level[])
2:     treeDepth ← 0;
3:     initialize empty queue Q;
4:     for all u ∈ V1 do
5:         if next[u] = ∅ then
6:             level[u] ← 1;
7:             Q.push(u);
8:         end if
9:     end for
10:    while not Q.isEmpty() do
11:        u ← Q.pop();
12:        for all x ∈ N(u) do
13:            if prev[x] = ∅ then
14:                treeDepth ← level[u];
15:                continue;
16:            end if
17:            if prev[x] = u then
18:                continue;
19:            end if
20:            if level[prev[x]] = 0 then
21:                level[prev[x]] ← level[u] + 1;
22:                Q.push(prev[x]);
23:            end if
24:        end for
25:        if treeDepth ≠ 0 then
26:            break;
27:        end if
28:    end while
29:    for all u ∈ V1 do
30:        if level[u] > treeDepth then
31:            level[u] ← 0;
32:        end if
33:    end for
34:    return (treeDepth ≠ 0);
35: end function
```

The first time we encounter a free vertex in V_2, we determine the last useful level in the current phase and we store it in the variable *treeDepth*. From that, we can also determine the length of a shortest augmenting path in the current phase (which is $2 \cdot \mathit{treeDepth} - 1$). We do not need to continue the BFS deeper than that level and the vertices that remain

in the queue belong either to the same level or to the next one. The vertices of the same level may still be used in the searching step to find other shortest augmenting paths, since we have already assigned their correct level. On the other hand, vertices that have already been assigned level $treeDepth + 1$ correspond to longer paths, so we reset their level to 0.

We define an array *next* for every $v \in V_1$, such that $next[v] = u$ if and only if the edge $\{v, u\}$ belongs to the current matching; otherwise, $next[v] = \varnothing$. Similarly, for every $u \in V_2$, we define $prev[u] = v$ if and only if the edge $\{u, v\}$ belongs to the current matching; otherwise, $prev[u] = \varnothing$. Thus, the arrays *next* and *prev* represent the current matching and are updated whenever we find an augmenting path and alternate its edges.

Theorem 2.2.6. *Algorithm 2.3 correctly computes the array level of the layered network of the current matching M. More precisely, if it returns* FALSE*, then no M-augmenting path exists in G. Otherwise, for every vertex $v \in V_1$, we have $level[v] \neq 0$ if and only if there exists an M-alternating path from a free vertex of V_1 to v. Moreover, if $level[v] \neq 0$, then the length of a shortest such path is $2 \cdot level[v] - 2$.*

Proof. For every free vertex $v \in V_1$, the algorithm sets $level[v] = 1$, and the corresponding alternating path has length 0. Now suppose that a vertex $u \in V_1$ is popped from the queue and that the statement already holds for u. Let $x \in V_2$ be a neighbor of u.

If $prev[x] = \varnothing$, then extending a shortest alternating path to u by the edge $\{u, x\}$ gives an M-augmenting path of length $2 \cdot level[u] - 1$. If $prev[x] = u$, then $\{u, x\}$ is a matched edge and cannot be used to continue an alternating path. Otherwise, if $prev[x] = w \neq u$ and $level[w] = 0$, then by appending the unmatched edge $\{u, x\}$ and the matched edge $\{x, w\}$ to a shortest alternating path ending at u, we obtain an alternating path from a free vertex of V_1 to w of length $(2 \cdot level[u] - 2) + 2 = 2 \cdot (level[u] + 1) - 2$. Hence, the algorithm correctly sets $level[w] = level[u] + 1$. Since vertices are processed in BFS order, this is the minimum possible alternating distance to w.

Therefore, for every vertex $v \in V_1$ with $level[v] \neq 0$, the value $level[v]$ is exactly the level of v in the layered network, and the length of a shortest alternating path from a free vertex of V_1 to v is $2 \cdot level[v] - 2$.

Finally, if the algorithm returns FALSE, then no free vertex of V_2 is reached during the BFS. Therefore, there is no M-augmenting path in G. Indeed, if such a path existed, then the vertex just before its free endpoint on a shortest such path would lie in the layered network, and the free vertex in V_2 would also be reached. □

Corollary 2.2.7. *If Algorithm 2.3 returns* TRUE*, then the length of a shortest M-augmenting path is $2 \cdot treeDepth - 1$.*

The searching step, Algorithm 2.4, is almost the same as Algorithm 2.1, so we will omit the proof. This time, we move across the layered network based on the *level* array. We also use an array *ptr* for all vertices in V_1. This array, holds for each vertex a counter that indicates the first neighbor that has not been checked yet. Hence, if we return to the same vertex in the same phase, we do not examine again the neighbors that have already returned FALSE. In particular, if $ptr[u]$ reaches the end of the adjacency list of u, then no augmenting path can be found through u in the current phase.

We could of course combine the two *if* statements into one.

Algorithm 2.4 DFS Phase of Hopcroft-Karp

Require: Bipartite graph $G = (V_1 \cup V_2, E)$, vertex $u \in V_1$, arrays *next*, *prev*, *level*, *ptr*
Ensure: Returns TRUE if a shortest augmenting path is found from u

```
1: function DfsHopcroftKarp(int u, int next[], int prev[], int level[], int ptr[])
2:     for ptr[u] to |N(u)| − 1 do
3:         x ← the ptr[u]-th neighbor of u;
4:         if prev[x] = ∅ then
5:             prev[x] ← u;
6:             next[u] ← x;
7:             ptr[u] ← |N(u)|;
8:             return TRUE;
9:         end if
10:        if level[prev[x]] = level[u] + 1 and DfsHopcroftKarp(prev[x], next, prev, level,
   ptr) then
11:            prev[x] ← u;
12:            next[u] ← x;
13:            ptr[u] ← |N(u)|;
14:            return TRUE;
15:        end if
16:    end for
17:    return FALSE;
18: end function
```

Algorithm 2.5 and lines 9-21 run the phases of the algorithm. In each phase, we call Algorithm 2.3 (line 9) to construct the layered network, find all augmenting paths of this graph by calling Algorithm 2.4 for each free vertex of V_1 (lines 13-17), and repeat the whole process. Note that we have to initialize the *ptr* array to 0 before starting the search, because all vertices will check their first neighbor first. We also have to initialize the *level* array to zero before building the layered network. The set *matching* contains the edges of the maximum matching.

Algorithm 2.5 Hopcroft–Karp Algorithm

Require: Bipartite graph $G = (V_1 \cup V_2, E)$
Ensure: Array *matching* contains the edges of the maximum matching

```
1:  function HopcroftKarp( )
2:      for all u ∈ V1 do
3:          next[u] ← ∅;
4:          level[u] ← 0;
5:      end for
6:      for all x ∈ V2 do
7:          prev[x] ← ∅;
8:      end for
9:      while BfsLayeredNetwork(next, prev, level) do
10:         for all u ∈ V1 do
11:             ptr[u] ← 0;
12:         end for
13:         for all u ∈ V1 do
14:             if next[u] = ∅ then
15:                 DfsHopcroftKarp(u, next, prev, level, ptr);
16:             end if
17:         end for
18:         for all u ∈ V1 do
19:             level[u] ← 0;
20:         end for
21:     end while
22:     matching ← ∅;
23:     for all u ∈ V1 do
24:         if next[u] ≠ ∅ then
25:             matching ← matching ∪ {{u, next[u]}};
26:         end if
27:     end for
28:     return matching;
29: end function
```

It is evident that constructing the layered network takes no more than $O(n + m)$. Moreover, lines 13-17 take no more than $O(n + m)$ as each edge is examined at most once (with the help of *ptr* array) and as a result when vertex returns FALSE, we are not going to process it again in the current phase.

Lemma 2.2.8. *Each phase, that is calculating the maximal set of vertex-disjoint shortest M-augmenting paths and take the symmetric difference with current matching M takes no more than $O(n + m)$ time.*

We shall show that we need at most $O(\sqrt{n})$ phases, but before that, we have to prove some important lemmas.

Lemma 2.2.9. *Let M and M' be two matchings of a graph G, with $|M'| > |M|$. Then the graph $M \triangle M'$ contains at least $|M'| - |M|$ vertex-disjoint M-augmenting paths.*

Proof. We follow the same logic as in Theorem 2.2.1. We consider the graph $M^* = M \triangle M'$. We can see that the graph contains components, each of which is an alternating path

or an even-length cycle. Cycles have the same number of edges from M and $M^{'}$, and the same holds for alternating paths of even length. Each component that represents an augmenting path has one more edge from one matching than from the other. In fact, every M-augmenting path has one more edge from $M^{'}$ than M. From $|M^{'}| > |M|$, we can see that we have more M-augmenting paths than $M^{'}$-augmenting paths, and there must be at least $|M^{'}| - |M|$ such paths. This completes the proof. □

The following is a special case of Lemma 2.2.9.

Lemma 2.2.10. *Given a matching M and a maximum matching M^*, the graph contains $|M^*| - |M|$ vertex-disjoint M-augmenting paths.*

Lemma 2.2.11. *Let $P = \{P_1, P_2, \cdots, P_t\}$ be a maximal set of vertex-disjoint shortest M-augmenting paths with length x that Algorithm 2.5 calculates in one phase and $M^{'} = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t) = M \triangle P_1 \triangle P_2 \triangle \cdots \triangle P_t$ with Q be an $M^{'}$-augmenting path. The set $R = (P_1 \cup P_2 \cup \cdots \cup P_t) \triangle Q$ has at least $x(t+1)$ edges.*

Proof. From $M^{'} = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t)$, we have

$$P_1 \cup P_2 \cup \cdots \cup P_t = M \triangle M^{'}$$

That is,

$$R = (M \triangle M^{'}) \triangle Q = M \triangle (M^{'} \triangle Q)$$

As Q is an $M^{'}$-augmented path, $M^{'} \triangle Q$ has size of $|M^{'}| + 1 = |M| + t + 1$ (plus $t + 1$ because it will get $t + 1$ more edges from alternating the paths of P and the path Q). By Lemma 2.2.9, there is at least $|M^{'} \triangle Q| - |M| = t + 1$ M-augmenting paths. Because every M-augmenting path has length at least x, it follows that R has at least $x(t + 1)$ edges. □

Lemma 2.2.12. *The length of the shortest augmenting path increases in every phase.*

Proof. Let's assume that the current matching is M and denote by $P = \{P_1, P_2, \cdots, P_t\}$ the maximal set of vertex-disjoint shortest M-augmenting paths of length x in one phase of Algorithm 2.5. Moreover, suppose that we take $M^{'} = M \triangle (P_1 \cup P_2 \cup \cdots \cup P_t)$ (the matching that we get at the end of phase) and let Q be an $M^{'}$-augmenting path. Obviously, Q has a common vertex v with at least one path from P, otherwise, it would mean that Q is also in M and either $|Q| > x$ (in this case we are done) or it would contradict the maximality of the set. Let $R = (P_1 \cup P_2 \cup \cdots \cup P_t) \triangle Q$. From Lemma 2.2.11, R has at least $x(t+1)$ edges. Let $P_i \in P$ such that $v \in P_i$. We observe that v is incident with a matched edge in $M^{'}$ because at the end of the phase we alternate the edges of each path in P. The endpoints of Q do not belong to $M^{'}$ because they are free, so it is guaranteed that v is an internal vertex of Q and therefore it is also incident with a matched edge in $M^{'}$. Obviously, it's impossible for v to be incident with two different matched edges from $M^{'}$, so P_i and Q have at least one edge from $M^{'}$ in common. It follows that

$$xt + x \leqslant |R| \leqslant |P_1 \cup P_2 \cup \cdots \cup P_t \cup Q| \leqslant xt + |Q| - 1$$

and hence $x + 1 \leqslant |Q|$. □

Lemma 2.2.13. *Algorithm 2.5 performs at most $O(\sqrt{n})$ phases.*

Proof. Assume that we have already performed $\sqrt{n}$ phases and the current matching is M. From Lemma 2.2.10, we know that there exist exactly $|M^*| - |M|$ vertex-disjoint M-augmenting paths for a maximum matching M^*. From Lemma 2.2.12 all these augmenting paths will have length at least $\sqrt{n}$, so there cannot be more than $\sqrt{n}$ (we would need more than n vertices) of them, which means that, $|M^*| - |M| \leqslant \sqrt{n}$. It follows from this that we can't perform more than $\sqrt{n}$ phases from that point. We conclude that the total phases can't be more than $2\sqrt{n} = O(\sqrt{n})$. □

From Lemma 2.2.8 and Lemma 2.2.13 we get the complexity of Hopcroft-Karp algorithm.

Theorem 2.2.14. *Algorithm 2.5 returns a maximum matching in $O((n+m)\sqrt{n})$ time.*

Maximum Bipartite Matching from Max-Flows We can reduce the bipartite maximum matching problem to the maximum flow problem by adding two extra nodes to the initial graph. The first vertex is called the source and will point to all vertices in V_1 with directed edges, and the second vertex is called the sink, such that all vertices from V_2 point to it. Moreover, all the edges of the initial graph will turn into directed edges, pointing to V_2 from a vertex of V_1. Assuming capacities of one for each edge (so there exists an integral maximum flow, that is every edge will have flow of 0 or 1), the maximum flow from source to sink is equal to the maximum bipartite matching. Actually, it is easy to see this because all paths that carry flow from source to sink should be edge-disjoint, and each path will have exactly one edge from V_1 to V_2, so no other path can use this edge again. We can get a maximum flow in the resulting graph from a maximum matching in the initial graph by just taking all the edges of matching and picking the edge from source to the V_1 vertex, and the edge from V_2 to the sink.

Consequently, someone could make the above reduction and use a famous flow algorithm, like Dinic [Din70] to solve the maximum bipartite problem with $O(n^2m)$. It turns out, that using some kind of the arguments we talked before, that Dinic's algorithm will perform with the same complexity as Hopcroft-Karp algorithm.

Recent Research Maximum flow is one of the most well-known problems in computer science and has been studied extensively over the years. Finding faster algorithms for the maximum flow problem, and and especially for the minimum-cost flow problem (to which maximum flow can be reduced), could lead to breakthroughs in many areas. For a long time, algorithms based primarily on combinatorial methods were far from the almost-linear-time bounds achieved by more recent continuous techniques. In 2023, a deterministic $m^{1+o(1)}$-time algorithm for exact maximum flow and minimum-cost flow was discovered [BCK+23]. This line of work also implies an $m^{1+o(1)}$-time algorithm for the maximum bipartite matching problem. On the other hand, randomized combinatorial algorithms were later obtained independently for exact maximum flow [BBST24] and for maximum bipartite matching [CK24].

2.3 Gale–Shapley Algorithm

2.4 Applications and Further Matching Models

Chapter 3

Polyhedral Approaches to Matching Problems

References

[BBST24] Aaron Bernstein, Joakim Blikstad, Thatchaphol Saranurak, and Ta-Wei Tu. Maximum flow by augmenting paths in $n^{2+o(1)}$ time. In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 2056–2077, 2024.

[BCK+23] Jan Van Den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A deterministic almost-linear time algorithm for minimum-cost flow. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 503–514, 2023.

[Ber57] Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences*, 43(9):842–844, 1957.

[CK24] Julia Chuzhoy and Sanjeev Khanna. Maximum bipartite matching in ⊠2+⊠(1) time via a combinatorial algorithm. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, page 83–94. Association for Computing Machinery, 2024.

[Din70] Yefim A. Dinitz. Algorithm for solution of a problem of maximum flow in networks with power estimation. *Soviet Mathematics Doklady*, 11:1277–1280, 1970.

[Din06] Yefim Dinitz. Dinitz' algorithm: The original version and even's version. In Oded Goldreich, Arnold L. Rosenberg, and Alan L. Selman, editors, *Theoretical Computer Science: Essays in Memory of Shimon Even*, pages 218–240. Springer, Berlin, Heidelberg, 2006.

[HK73] John E Hopcroft and Richard M Karp. An n^5/2 algorithm for maximum matchings in bipartite graphs. *SIAM Journal on computing*, 2(4):225–231, 1973.

[Pet91] Julius Petersen. Die theorie der regulären graphs, 1891.